Day 2 : Snowflake Objects and SQL Commands

By Gurleen Kaur



Day 2 Agenda

01

Snowflake Objects

Databases, schemas, tables, views, and stored procedures

02

SQL Commands

DDL, DML, and query constructs for effective data manipulation

03

Data Loading

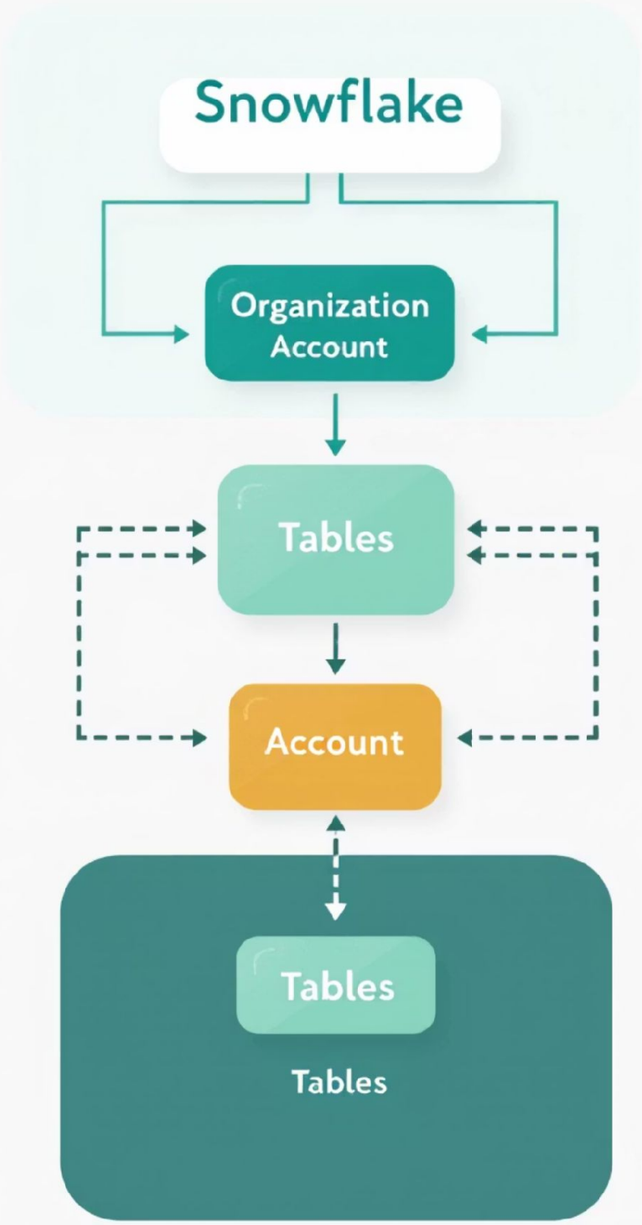
Loading data from AWS S3 using COPY INTO and Snowpipe

04

Stage Management

Internal and external stages with file format specifications

By the end of this session, you'll understand the core components that make Snowflake a powerful platform for modern data engineering and analytics workloads.



Snowflake Object Hierarchy

Organization & Account

The top-level container that holds all your Snowflake resources. Organizations can contain multiple accounts for dev/test/prod environments.

Databases

Logical containers for organizing schemas. Unlike traditional databases, Snowflake databases are lightweight and you can create many without performance overhead.

Schemas

Namespaces within databases that organize database objects like tables, views, and procedures. A single database can contain hundreds of schemas.

Objects

Tables, views, stored procedures, and other database objects that store or process your data.

Databases, Schemas and Tables

Databases

Top-level containers that organize related schemas.

```
CREATE DATABASE sales_data;USE DATABASE sales_data;
```

Schemas

Logical groupings of database objects.

```
CREATE SCHEMA sales_data.transactions;USE SCHEMA sales_data.transactions;
```

Table Types

- **Permanent:** Default type, persists until explicitly dropped
- **Transient:** No Fail-safe period, lower cost for temporary workloads
- **Temporary:** Session-specific, automatically dropped when session ends

```
CREATE TRANSIENT TABLE customer_staging (  id INTEGER,  name VARCHAR(100),  email VARCHAR(255));
```

Choose the right table type based on data importance, retention needs, and cost considerations.

Working with Database Objects

```
-- Create a databaseCREATE DATABASE sales_data;-- Create a schemaCREATE SCHEMA sales_data.transactions;-- List schemas in databaseSHOW SCHEMAS IN DATABASE sales_data;-- Set context (important!)USE DATABASE sales_data;USE SCHEMA transactions;
```

Working Efficiently with Objects

- Always set your session context to avoid fully qualifying object names in every statement
- Use descriptive naming conventions (e.g., `db_env_purpose`)
- Consider data isolation needs when planning your hierarchy
- Keep development, testing, and production in separate databases

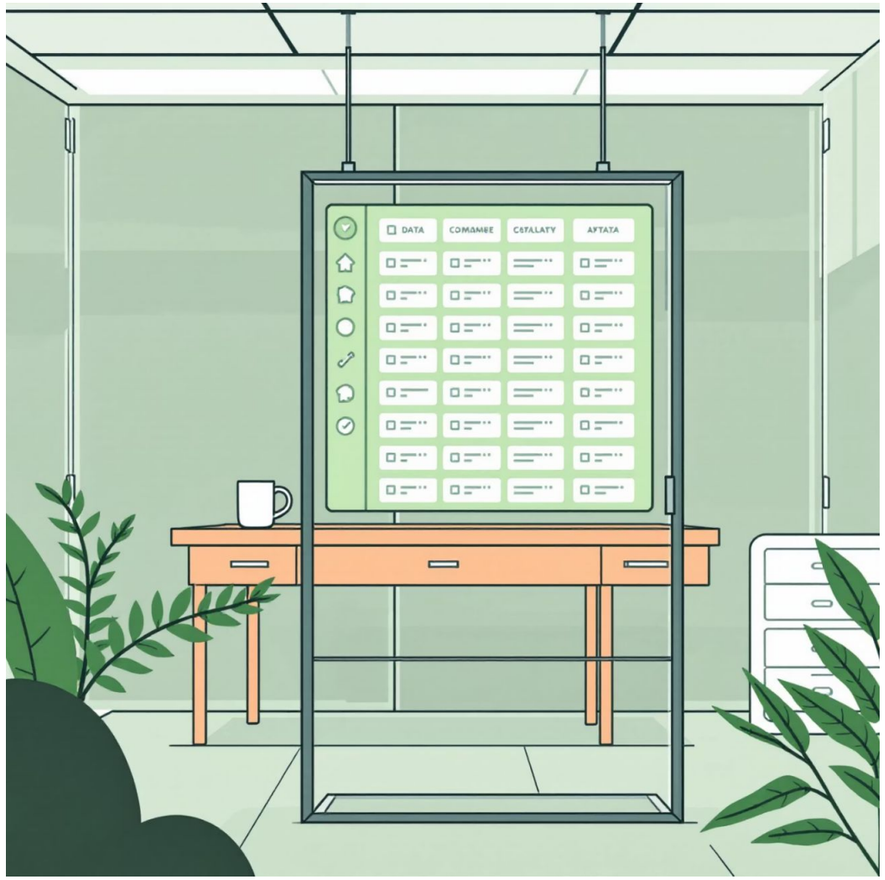
Pro tip: Use `INFORMATION_SCHEMA` views to programmatically explore object metadata instead of `SHOW` commands.

Permanent Tables

Key Characteristics

- Default table type in Snowflake
- Persist until explicitly dropped
- Full data protection with Time Travel and Fail-safe
- Higher storage costs due to retention features
- Support for clustering keys to optimize query performance

```
CREATE TABLE customer_orders (  order_id NUMBER,  customer_id
VARCHAR(20),  order_date DATE,  amount DECIMAL(12,2))CLUSTER BY
(order_date);
```



Permanent tables are ideal for critical business data that requires point-in-time recovery and protection against accidental deletion.

Transient & Temporary Tables

Transient Tables

- No Time Travel or Fail-safe protection
- Lower storage costs (30-50% cheaper)
- Persist until explicitly dropped
- Ideal for intermediate ETL results

```
CREATE TRANSIENT TABLE staging_dataAS SELECT * FROM raw_data;
```

Temporary Tables

- Exist only for the duration of the session
- Automatically dropped when session ends
- Only visible to the creating session
- Perfect for query-specific transformations

```
CREATE TEMPORARY TABLE session_metricsAS SELECT user_id, COUNT(*) FROM events GROUP BY user_id;
```

When working with large datasets, temporary tables can significantly improve query performance by materializing intermediate results. Transient tables strike a balance between cost and persistence.

Standard Views

What Are Standard Views?

Virtual tables defined by a query that dynamically retrieves data from underlying tables each time the view is referenced.

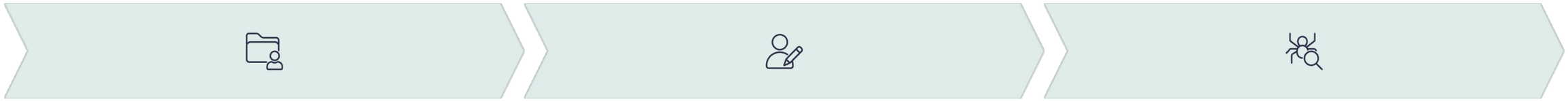
- No physical storage (query executed on-demand)
- Always returns current data
- Can join across multiple tables
- Simplifies complex queries
- Controls access to sensitive columns

```
CREATE VIEW customer_sales_summary ASSELECT   c.customer_name,  c.region,
SUM(o.amount) as total_salesFROM customers cJOIN orders o ON c.id =
o.customer_idGROUP BY c.customer_name, c.region;
```



Standard views can also be used to implement row-level security by including WHERE clauses that filter data based on the current user.

DDL, DML, and Query Constructs



Data Definition Language (DDL)

Commands that define and modify database structures

- CREATE, ALTER, DROP, TRUNCATE
- DESCRIBE, SHOW
- GRANT, REVOKE

Data Manipulation Language (DML)

Commands that manipulate data within database objects

- SELECT, INSERT, UPDATE, DELETE
- MERGE, COPY
- UNLOAD

Query Constructs

Advanced features for data analysis

- Window functions
- Common Table Expressions (CTEs)
- Semi-structured data functions (JSON, Parquet, Avro)

Snowflake supports the full ANSI SQL standard plus cloud-specific extensions, making it both familiar to SQL users and powerful for cloud data analytics.

Materialized Views

Key Features

- Pre-computed results stored physically
- Automatic incremental maintenance
- Significantly faster query performance
- Consume storage space (unlike standard views)
- Higher cost but much better performance

Best Use Cases

- Frequently accessed aggregations
- Dashboards with common metrics
- Complex joins that are reused
- When query latency is critical
- For improving BI tool performance

```
CREATE MATERIALIZED VIEW daily_sales_by_region ASSELECT   DATE_TRUNC('day', order_date) AS day,  region,  SUM(amount) AS daily_totalFROM ordersJOIN customers ON orders.customer_id = customers.idGROUP BY 1, 2;
```

Unlike standard views, materialized views in Snowflake are automatically and incrementally refreshed when their base tables change, ensuring data is always current without manual refreshes.

Stored Procedures

JavaScript Procedures

More flexible, can include control logic, API calls, and complex operations.

```
CREATE OR REPLACE PROCEDURE process_orders()  RETURNS STRING  LANGUAGE
JAVASCRIPT  AS  $$  var sql_command =     `INSERT INTO order_summary
SELECT date_trunc('month', order_date),            SUM(amount)     FROM
orders     WHERE processed = false     GROUP BY 1`;    var stmt =
snowflake.createStatement(             { sqlText: sql_command });  var
result = stmt.execute();    return "Processing complete";  $$;
```

SQL Procedures

Simpler to write, ideal for straightforward database operations.

```
CREATE OR REPLACE PROCEDURE update_inventory() RETURNS STRING LANGUAGE SQL
AS BEGIN UPDATE inventory i SET stock_level = i.stock_level -  (SELECT
SUM(quantity)  FROM order_items oi JOIN orders o ON oi.order_id = o.id
WHERE o.status = 'SHIPPED' AND o.processed = false AND oi.product_id =
i.product_id);  UPDATE orders SET processed = true WHERE status =
'SHIPPED'  AND processed = false;  RETURN 'Inventory updated'; END;
```

SQL Command Types

1

DDL (Data Definition Language)

- CREATE, ALTER, DROP, TRUNCATE, RENAME
- Define and modify database structures
- CREATE or REPLACE to update object definitions
- UNDROP to recover accidentally dropped objects

2

DML (Data Manipulation Language)

- INSERT, UPDATE, DELETE, MERGE
- Modify data stored in tables
- MERGE for upsert operations
- Multi-table INSERT with subqueries

3

DQL (Data Query Language)

- SELECT statements for data retrieval
- Complex joins, window functions
- Snowflake-specific functions
- Semi-structured data queries (FLATTEN, PARSE_JSON)

Snowflake extends standard SQL with specialized functions for working with semi-structured data, geospatial data, and time series analysis. These extensions make complex data processing significantly easier compared to traditional databases.

Snowflake-Specific SQL Features

Semi-structured Data

```
-- Query JSON dataSELECT   raw_data:customer.name::STRING,
raw_data:customer.email::STRING,  raw_data:items[0].product_id::NUMBERFROM
json_orders;-- Flatten arraysSELECT value:idFROM json_orders,LATERAL FLATTEN(input =>
raw_data:items);
```

Zero-Copy Cloning

```
-- Clone a table (no physical copy)CREATE TABLE orders_backupCLONE orders;-- Clone a
database at a point in timeCREATE DATABASE sales_backupCLONE sales AT(TIMESTAMP =>
'2023-06-15 13:00:00'::TIMESTAMP);
```

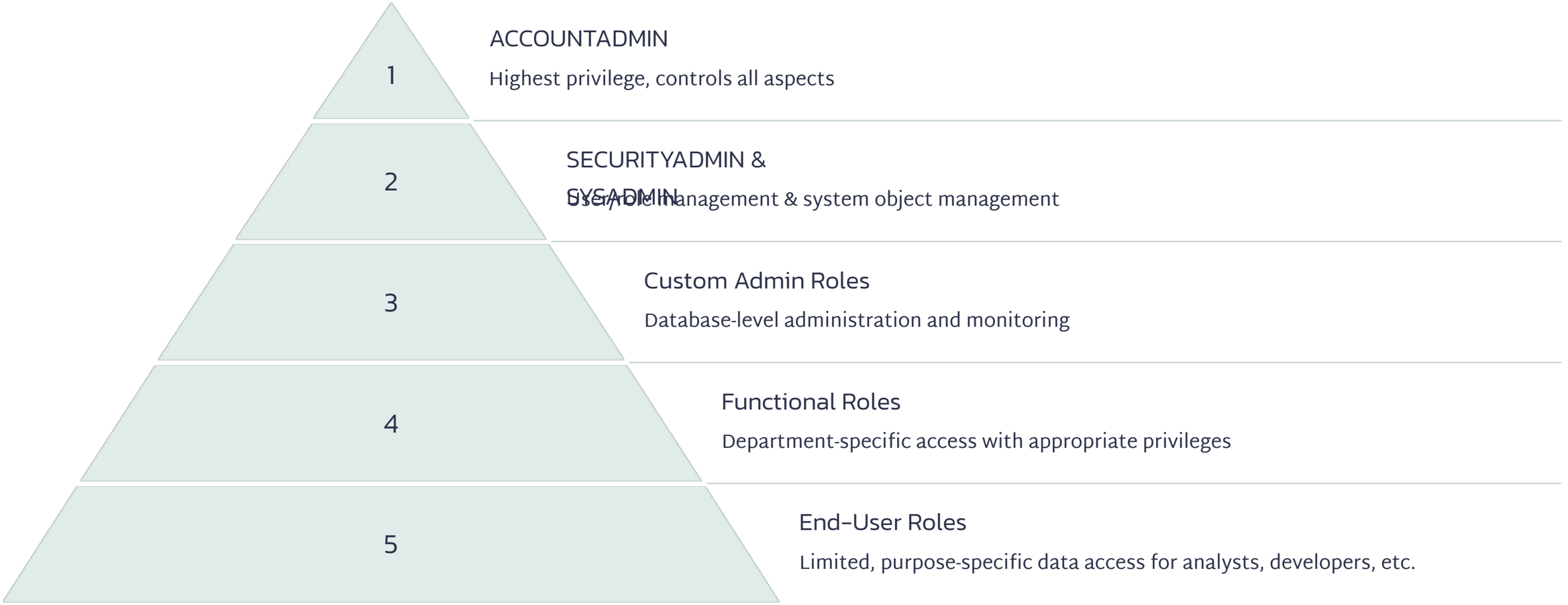
Data Sharing

```
-- Create a shareCREATE SHARE marketing_share;-- Grant usage on databaseGRANT USAGE ON
DATABASE analytics TO SHARE marketing_share;-- Add consumer accountALTER SHARE
marketing_share ADD ACCOUNTS = abc123.us-east-1;
```

Time Travel

```
-- Query data as it was 2 hours agoSELECT * FROM orders AT(OFFSET => -60*60*2);--
Restore a dropped tableUNDROP TABLE accidentally_dropped;-- Restore a table to a
specific timeCREATE TABLE orders_fixed CLONE ordersBEFORE (STATEMENT =>
'019abc58-0001-0001');
```

User Roles and Access Control



Snowflake implements Role-Based Access Control (RBAC) with hierarchical role inheritance. Each user can have multiple roles and switch between them as needed.

Implementing Access Control

Role Management

```
-- Create functional rolesCREATE ROLE
marketing_analyst;CREATE ROLE finance_analyst;-- Grant access
to rolesGRANT ROLE marketing_analyst TO USER jane_doe;-- Role
hierarchyGRANT ROLE marketing_analyst TO ROLE
department_lead;-- Set active roleUSE ROLE marketing_analyst;
```

Object Privileges

```
-- Database-level accessGRANT USAGE ON DATABASE sales_data TO
ROLE marketing_analyst;-- Schema-level accessGRANT USAGE ON
SCHEMA sales_data.transactions TO ROLE marketing_analyst;--
Object-level privilegesGRANT SELECT ON TABLE
sales_data.transactions.orders TO ROLE
marketing_analyst;GRANT SELECT ON VIEW
sales_data.reporting.dashboard_data TO ROLE
marketing_analyst;
```

⚠ Never assign ACCOUNTADMIN to everyday users. Create custom admin roles with limited scope for specific administrative functions instead.

Best Practices & Key Takeaways

Object Organization

Follow a consistent naming convention and organize objects logically across databases and schemas. Consider using environment prefixes (DEV_, TEST_, PROD_).

Table Choices

Use permanent tables for critical data, transient tables for ETL, and temporary tables for session-specific work. Apply clustering keys to large tables with selective filters.

View Strategy

Use standard views for data security and simplifying complex joins. Reserve materialized views for performance-critical aggregations accessed frequently.

Security Model

Implement least-privilege access with functional roles. Use a role hierarchy that mirrors your organization structure. Regularly audit access patterns.

Remember that Snowflake separates storage and compute, which should influence your design decisions. Objects that consume storage (tables, materialized views) incur ongoing costs, while compute resources scale independently based on workload needs.

For complex operations, combine stored procedures with task scheduling to automate workflows and ensure consistency in your data processing pipelines.

User Roles and Access Control

Snowflake implements a role-based access control system that determines what actions users can perform on which objects. Roles can be assigned to users and hierarchically organized.

System-Defined Roles

- **ACCOUNTADMIN:** Top-level administration role with full control
- **SECURITYADMIN:** Manages users, roles, and privileges
- **SYSADMIN:** Creates and manages databases and warehouses
- **PUBLIC:** Automatically granted to all users

Custom Roles

- Create role hierarchies for specific business needs
- Implement least privilege principle
- Control access at database, schema, or object level
- Apply column-level security for sensitive data

- Create custom role and grant privileges
- CREATE ROLE data_analyst;GRANT USAGE ON DATABASE analytics TO ROLE data_analyst;GRANT USAGE ON SCHEMA analytics.marketing TO ROLE data_analyst;GRANT SELECT ON TABLE analytics.marketing.campaign_performance TO ROLE data_analyst;

Module 4: Data Loading in Snowflake Using AWS S3

Efficient strategies for bringing your data into Snowflake

Snowflake and S3 Integration Architecture

Snowflake and AWS S3 create a powerful data pipeline through secure, high-throughput connections:

- **Storage Integration:** Secure, credential-free authentication between Snowflake and S3
- **Internal vs. External Stages:** S3 buckets can be referenced as external stages

- **Secure Access:** IAM roles with least privilege principles
- **Data Transfer:** Compute resources in Snowflake's virtual warehouses pull data directly from S3
- **Metadata Catalog:** Snowflake maintains table metadata separate from storage



External Stages: Your Gateway to S3 Data

Creating an External Stage

```
CREATE STAGE my_s3_stage  URL = 's3://my-bucket/path/'
CREDENTIALS = (AWS_ROLE = 'arn:aws:iam::account:role/myrole')  ENCRYPTION = (TYPE = 'AWS_SSE_KMS' KMS_KEY_ID = 'key_id');
```

Links Snowflake to your S3 bucket with secure role-based access

Defining File Formats

```
CREATE FILE FORMAT my_csv_format  TYPE = 'CSV'
FIELD_DELIMITER = ','
SKIP_HEADER = 1  NULL_IF = ('NULL', 'null')
EMPTY_FIELD_AS_NULL = TRUE;
```

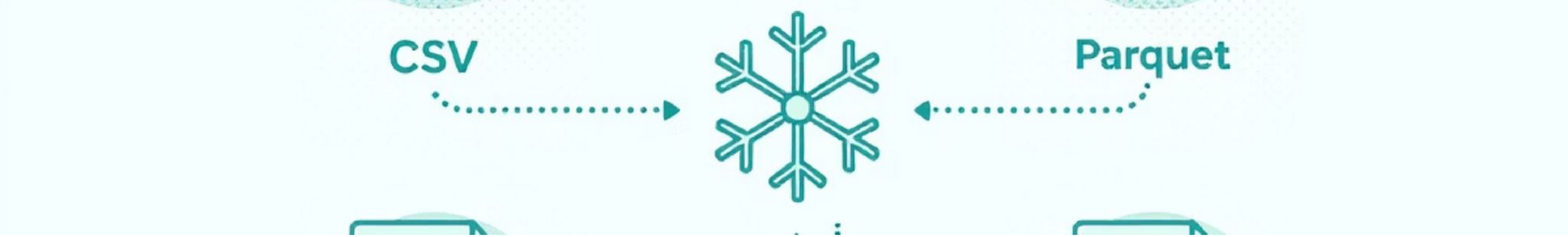
Tells Snowflake how to interpret data structures in your files

Listing Files in Stage

```
LIST @my_s3_stage/daily_feeds/;
```

Verifies file availability before loading operations

External stages provide the foundation for all data loading operations and can be reused across multiple workflows.



File Format Configuration Deep Dive

Supported File Types

- **CSV/TSV:** Delimiter options, quote handling, escape characters
- **JSON:** Object or array formats with path extraction
- **Parquet:** Column-oriented with schema inference
- **Avro:** Row-based binary format with schema registry support
- **ORC:** Optimized Row Columnar for Hadoop ecosystems
- **XML:** With XPath expressions for element selection

Advanced Format Options

- **Compression:** AUTO, GZIP, BZ2, BROTLI, ZSTD, DEFLATE
- **Error Tolerance:** SKIP_FILE, CONTINUE, ABORT_STATEMENT
- **Date/Time Formats:** Custom parsing patterns
- **Encoding:** UTF8, UTF16, ISO-8859-1, etc.
- **Binary Format:** HEX, BASE64 encoding for binary data

> **Best Practice**
>
> Create separate file formats for each data source type and version them in your code repositories. File formats can be shared across multiple loading operations.

COPY INTO: The Workhorse of Batch Loading

```
COPY INTO sales_data  FROM @my_s3_stage/sales/  FILES = ('sales_20230101.csv', 'sales_20230102.csv')  FILE_FORMAT =
(FORMAT_NAME = my_csv_format)  PATTERN = '.*sales_[0-9]{8}\.csv'  ON_ERROR = 'CONTINUE'  VALIDATION_MODE = 'RETURN_ERRORS'
FORCE = FALSE;
```

Key Parameters

- **FILES vs. PATTERN:** Explicit list or regex pattern matching
- **FORCE:** Control file reloading behavior (deduplication)
- **ON_ERROR:** Error handling strategy
- **SIZE_LIMIT:** Controls maximum bytes processed

Runtime Transformations

- **Column Selection:** Load specific columns only
- **Type Casting:** Convert data types during load
- **Expressions:** Apply SQL functions during ingestion
- **COPY_HISTORY:** Access load metadata for auditing

Parallel Loading

- **Warehouse Sizing:** X-Large+ for maximum throughput
- **File Sizing:** Aim for 100-250MB compressed files
- **File Count:** Multiple files load in parallel
- **SPLIT:** Break large files for better parallelization

Data Transformation During Load

Column-Level Transformations

```
COPY INTO customers(  id,  name,  email,   signup_date,  status,  loyalty_score)FROM (  SELECT     $1::integer,    INITCAP($2::string),    LOWER($3::string),
TO_DATE($4, 'MM/DD/YYYY'),    IFF($5 = 'A', 'ACTIVE', 'INACTIVE'),    ZEROIFNULL($6::number(3,1))  FROM @my_s3_stage/customers/)FILE_FORMAT = (FORMAT_NAME =
my_csv_format);
```

Snowflake can transform data on ingestion, reducing the need for separate ELT steps. This approach leverages the compute power of your virtual warehouse during the load process itself.

Tip: For complex transformations, consider using a separate transformation layer or Snowflake Streams and Tasks for a more maintainable architecture.

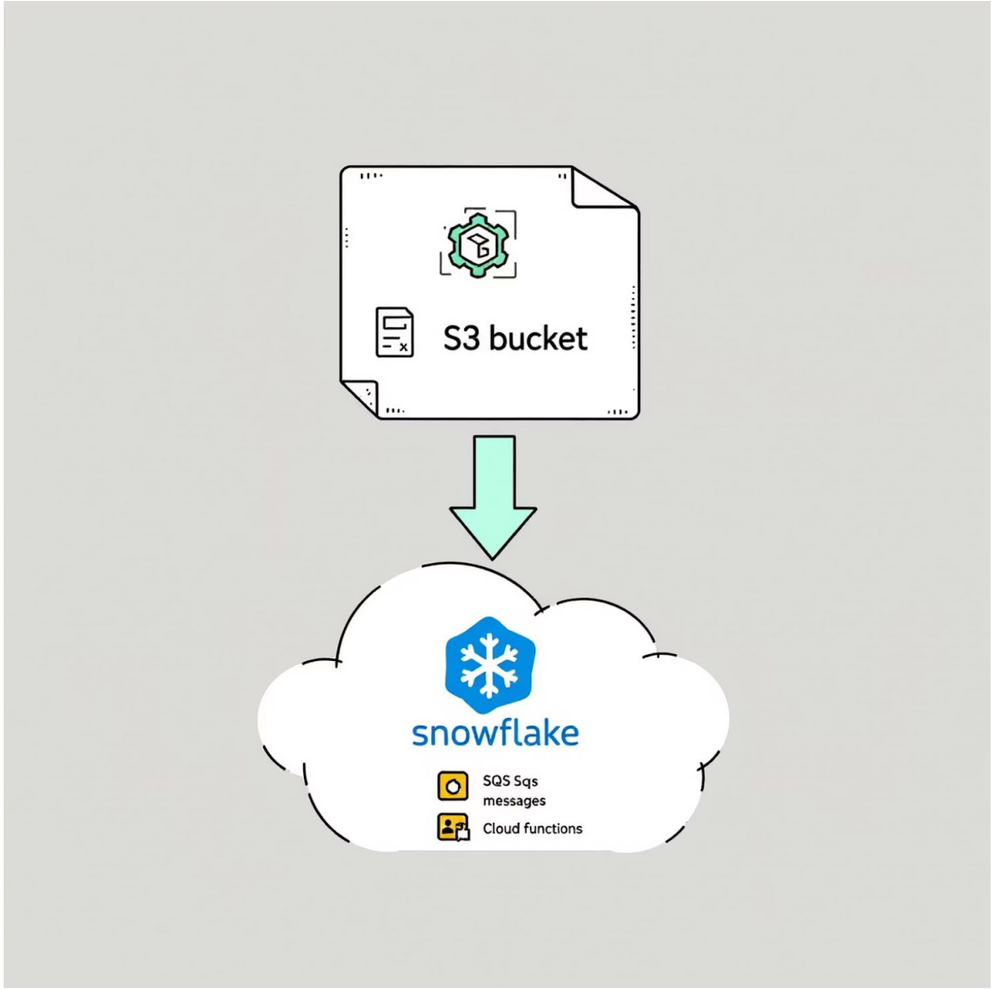
Snowpipe: Continuous Data Loading

Serverless, Near Real-Time Ingestion

Snowpipe provides automated, incremental data loading without managing virtual warehouses:

- **Event-Driven:** Triggered by S3 events via SQS notifications
- **Serverless:** Billed by compute seconds, not warehouse time
- **Auto-Ingest:** Fully automated pipeline with event notifications
- **REST API:** Programmatic control for custom integration
- **Manifest Files:** Track loading state for reliability

Ideal for streaming data, IoT sensors, clickstreams, and other continuous data sources.



Implementing Snowpipe

Create a Pipe Object

```
CREATE PIPE sales_data_pipe
AUTO_INGEST = TRUE  AS COPY
INTO sales_data  FROM
@my_s3_stage/sales_pipe/
FILE_FORMAT = (FORMAT_NAME =
my_csv_format);
```

The AUTO_INGEST parameter enables event-driven loading

Configure S3 Event Notifications

```
-- Get the SQS queue ARN for
your pipeDESC PIPE
sales_data_pipe;-- Configure in
AWS Console or CLI:-- 1. S3
bucket -> Properties ->
Events-- 2. Add SQS
notification for ObjectCreated
events-- 3. Use the ARN from
DESC PIPE command
```

Monitor Loading Progress

```
-- Check pipe statusSELECT *
FROM
TABLE(INFORMATION_SCHEMA.PIPE_U
SAGE_HISTORY(
DATE_RANGE_START=>DATEADD('hour
s',-24,CURRENT_TIMESTAMP()),
PIPE_NAME=>'sales_data_pipe'));
```

After setup, files dropped in the configured S3 location will automatically trigger loading into Snowflake without manual intervention.

Choosing Between Batch and Continuous Loading

Use COPY INTO When

- Loading large historical datasets
- Performing complex transformations during load
- Requiring transaction control (all-or-nothing loading)
- Loading on a scheduled basis (hourly/daily/weekly)
- Needing direct control over warehouse resources

Use Snowpipe When

- Processing small files continuously (KB to low MB)
- Requiring near real-time data availability (minutes)
- Optimizing for cost with irregular data arrival
- Implementing event-driven architectures
- Avoiding warehouse management overhead

Hybrid Approach

Use Snowpipe for continuous ingestion during the day



Scheduled Backfill

Run COPY INTO for any missed files overnight

Monitoring

Track latency, errors, and volume metrics

Reconciliation

Compare source and target record counts daily

Performance Optimization Techniques

File Optimization

- **Size:** 100-250MB compressed files load fastest
- **Compression:** Always compress (GZIP, ZSTD preferred)
- **Partitioning:** Organize files by date/region for selective loading
- **Format:** Parquet/ORC outperform CSV/JSON for large datasets
- **Naming:** Consistent patterns enable automation

Loading Strategies

- **Parallel Loading:** More files = better parallelization
- **Warehouse Sizing:** Match to file count and complexity
- **Staging Tables:** Load raw, then transform in SQL
- **Microbatching:** Group small files for efficient loading
- **Avoid FORCE:** Use metadata to track loaded files

10x

Performance Gain

Optimizing file sizes and formats can yield up to 10x improvement in load times

60%

Cost Reduction

Proper warehouse sizing and scheduling can reduce compute costs significantly

99.9%

Reliability Target

Well-designed pipelines should achieve near-perfect data delivery

External Stages and File Formats

External Stages

References to external cloud storage locations where your source data resides.

```
CREATE STAGE my_s3_stage  URL='s3://my-bucket/data/'
CREDENTIALS=(AWS_KEY_ID='...' AWS_SECRET_KEY='...')
ENCRYPTION=(TYPE='AWS_SSE_S3');
```

You can also use AWS IAM roles for more secure authentication:

```
CREATE STAGE my_s3_stage  URL='s3://my-bucket/data/'
STORAGE_INTEGRATION = my_aws_int;
```

File Formats

Specifications that tell Snowflake how to interpret your data files.

```
CREATE FILE FORMAT my_csv_format  TYPE = 'CSV'
FIELD_DELIMITER = ','  SKIP_HEADER = 1  NULL_IF =
('NULL', 'null')  EMPTY_FIELD_AS_NULL = TRUE;
```

Supported types include:

- CSV, TSV, JSON, Avro, ORC, Parquet, XML
- Compressed files (GZIP)

Loading Data Using COPY INTO

The COPY INTO command is Snowflake's primary bulk data loading mechanism, optimized for performance and flexibility.

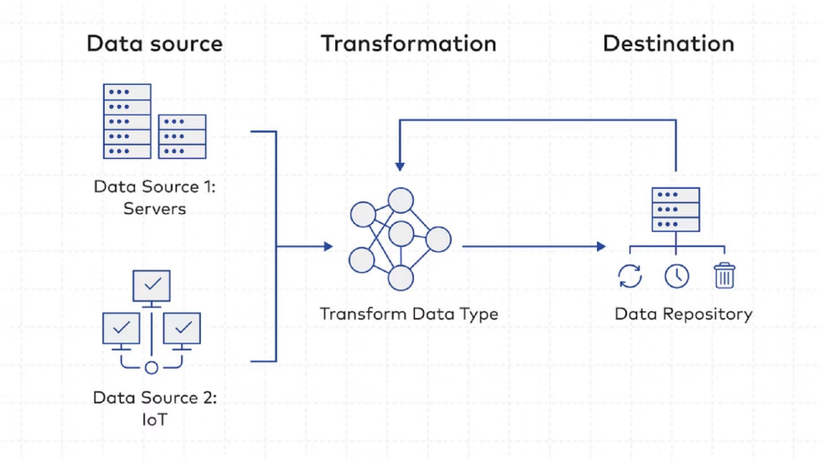
```
COPY INTO sales.transactionsFROM @my_s3_stage/transactions/FILES =
('transactions_2023_01.csv', 'transactions_2023_02.csv')FILE_FORMAT = (FORMAT_NAME =
my_csv_format)ON_ERROR = 'CONTINUE'VALIDATION_MODE = 'RETURN_ERRORS';
```

Key Features

- Parallel loading across compute nodes for maximum performance
- File pattern matching with wildcards
- Data transformation during load (COPY with SELECT)
- Error handling and validation options

Best Practices

- Use compressed files (GZIP, BZ2) for faster transfers
- Size files between 100-250 MB for optimal parallelism
- Use column lists to enforce schema expectations
- Consider using FORCE=TRUE for re-loads



Transforming Data During Load

Snowflake allows you to transform data on-the-fly during loading, eliminating the need for separate ETL processes.

```
COPY INTO sales.transactions(  transaction_id,   customer_id,  transaction_date,  amount,  status)FROM (  SELECT     $1,    $2,    TO_DATE($3,
'MM/DD/YYYY'),    TO_DECIMAL($4, 10, 2),    CASE WHEN $5 = 'C' THEN 'COMPLETED'          WHEN $5 = 'P' THEN 'PENDING'         ELSE 'UNKNOWN' END  FROM
@my_s3_stage/transactions/)FILE_FORMAT = (FORMAT_NAME = my_csv_format);
```

> This approach, known as "ELT" (Extract, Load, Transform), leverages Snowflake's compute resources for transformation rather than external tools, simplifying your data pipeline and reducing points of failure.

Automating Ingestion with Snowpipe

What is Snowpipe?

Snowflake's serverless, near real-time continuous data ingestion service.

- Automatically loads new files as they arrive in your stage
- Serverless compute - no warehouse required
- Pay only for the data processed
- Designed for frequent, smaller loads

Implementation

```
-- Create the pipeCREATE PIPE sales.transactions_pipe
AUTO_INGEST = TRUEASCOPY INTO sales.transactionsFROM
@my_s3_stage/transactions/FILE_FORMAT = (FORMAT_NAME =
my_csv_format);
```

When AUTO_INGEST=TRUE, Snowflake creates an SQS notification integration. Configure your S3 bucket to send notifications when new files arrive.

Snowpipe is ideal for applications requiring fresh data with minimal latency, such as real-time dashboards, operational reporting, and event-driven architectures.

Bulk vs. Continuous Loading

Bulk Loading (COPY INTO with Warehouse)

Best for:

- Large historical data loads
- Regular batch processing (daily, hourly)
- When data freshness can wait for batch windows
- Complex transformations during load

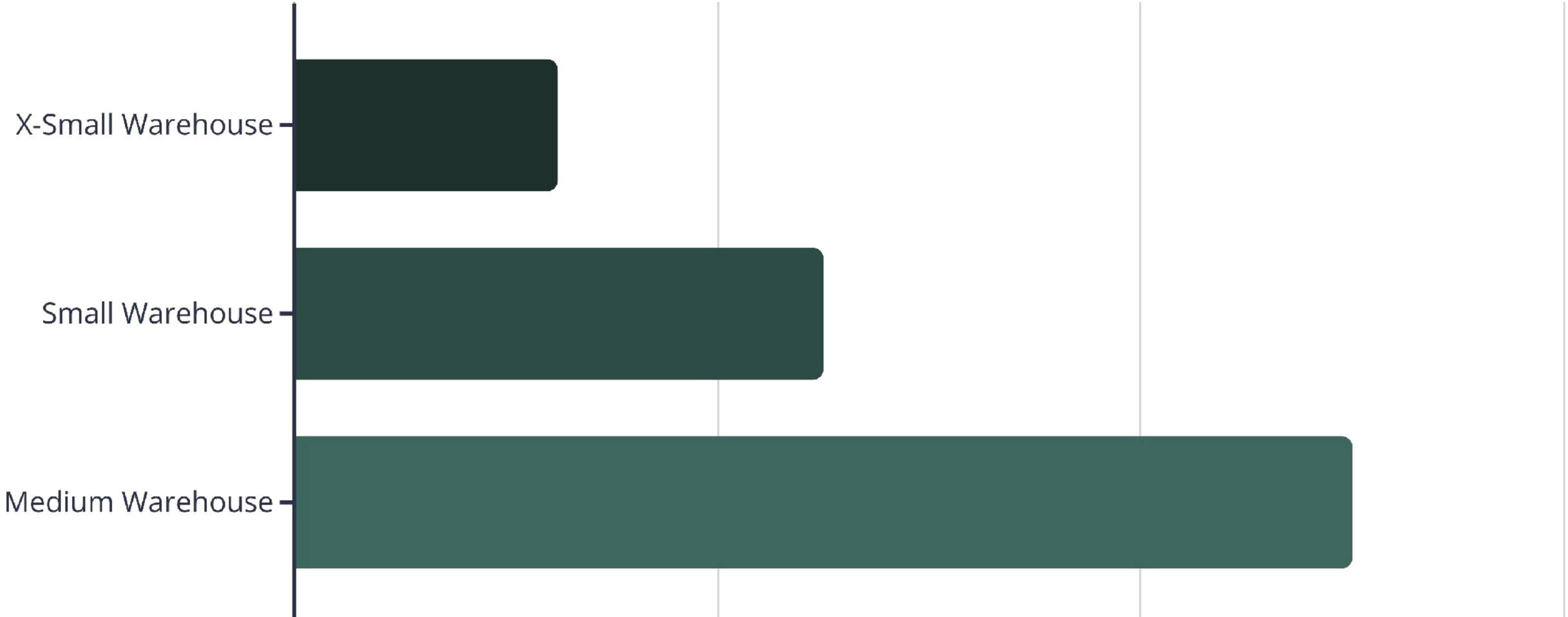
Uses a virtual warehouse for parallelized processing power

Continuous Loading (Snowpipe)

Best for:

- Continuous streams of smaller files
- Near real-time data availability requirements
- Serverless operations (no warehouse management)
- Microservices and event-driven architectures

Automatically processes files as they arrive in your stage



Module 5: Understanding Stages in Snowflake

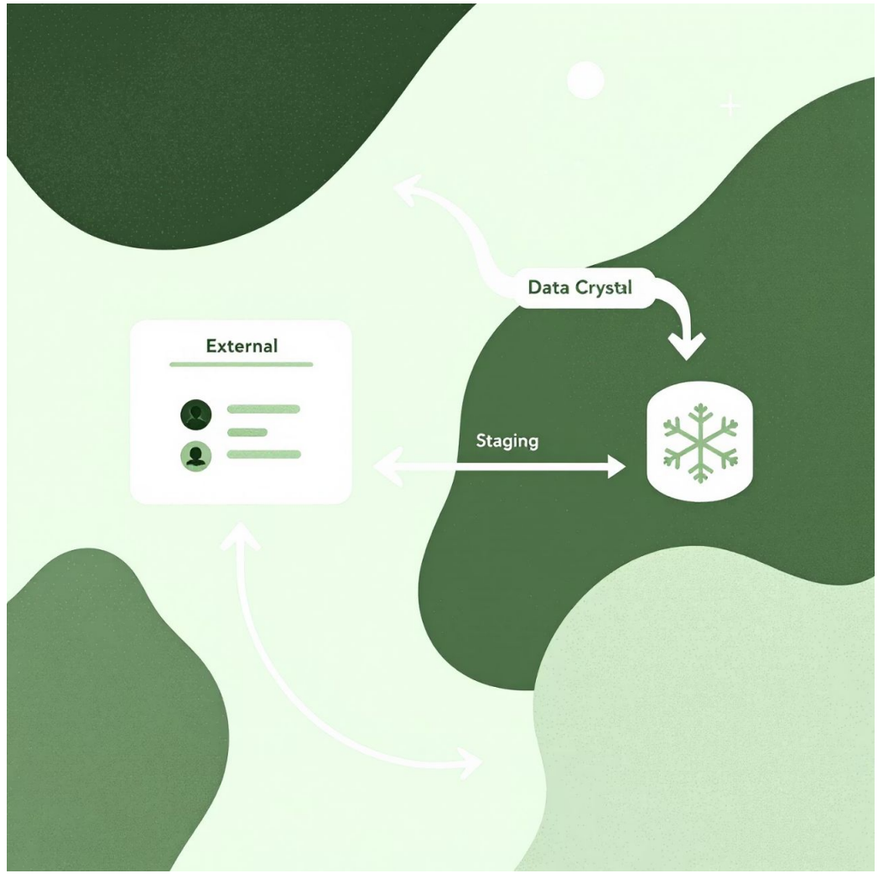
Mastering the staging areas for efficient data operations

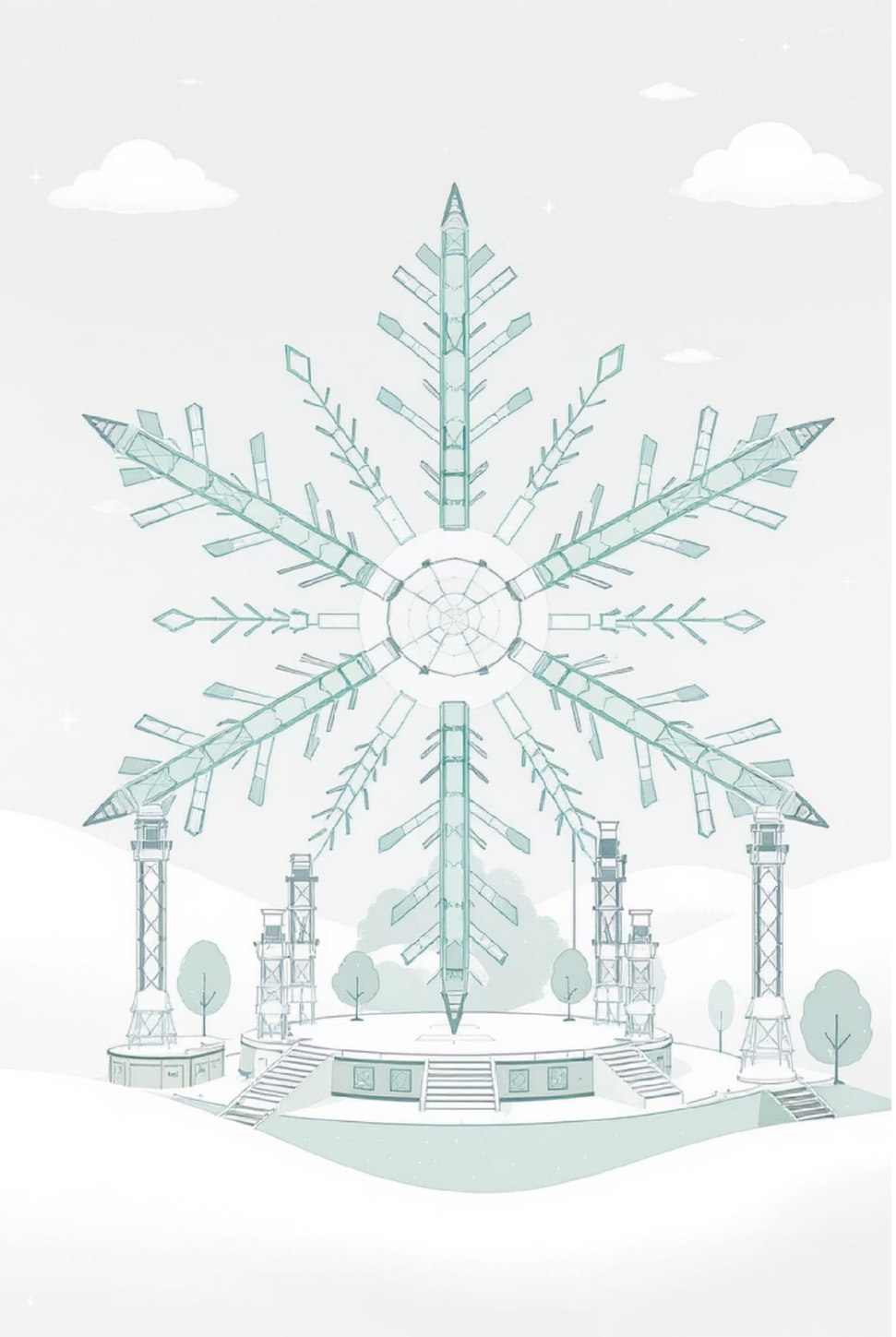
What Are Snowflake Stages?

Stages in Snowflake are **locations where data files are stored** before loading into or after unloading from Snowflake tables. They serve as the intermediate storage layer in the data loading/unloading process.

Stages are crucial components in your data pipeline architecture that enable:

- Structured data loading workflows
- Separation of storage and compute concerns
- Controlled data movement between systems
- Audit trails for data operations





Types of Stages in Snowflake

Internal Stages

Storage locations managed directly by Snowflake.

- **User stages:** @~/ (private to each user)
- **Table stages:** @%table_name (associated with specific tables)
- **Named stages:** @stage_name (accessible across the account)

External Stages

References to external cloud storage services.

- AWS S3 buckets
- Google Cloud Storage
- Microsoft Azure Blob Storage
- Require proper authentication setup

Stages serve as the intermediary locations where data resides before being loaded into Snowflake tables or after being unloaded from tables. They provide a structured way to organize your data movement operations.

Working with S3 External Stages

Creating an External Stage

```
Create storage integration (account admin only)CREATE STORAGE INTEGRATION s3_int  TYPE = EXTERNAL_STAGE  STORAGE_PROVIDER = S3  ENABLED = TRUE  STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::123456789012:role/snowflake-access’
  STORAGE_ALLOWED_LOCATIONS = ('s3://my-bucket/data/');-- Create the stage using the integrationCREATE STAGE marketing.campaign_data STORAGE_INTEGRATION = s3_int  URL = 's3://my-bucket/data/campaigns/'  FILE_FORMAT = (TYPE = 'PARQUET');
```

Key S3 Stage Features

- **Authentication options:**
  - Storage Integration (recommended)
  - AWS Key/Secret (legacy)
- **Security features:**
  - Encryption in transit
  - AWS SSE-S3/SSE-KMS support
  - Access restricted by allowed locations
- **Performance optimization:**
  - Regional co-location
  - Parallel operations

Using storage integrations provides better security by avoiding hardcoded credentials and leveraging IAM roles for authentication.

The Four Types of Snowflake Stages

1

Internal Stages

Managed entirely within Snowflake's storage. Best for temporary staging needs with automatic management. Storage is charged as part of your Snowflake account.

```
CREATE STAGE my_internal_stage;
```

2

External Stages

References external cloud storage (primarily S3). Ideal for persistent storage needs and cross-platform integration. Storage costs are handled by your cloud provider.

```
CREATE STAGE my_s3_stage URL='s3://bucket/path/'
```

3

User Stages

Automatically created for each user. Accessed with the `@~` prefix. Perfect for personal development work and ad-hoc operations.

```
PUT file:///path/to/local/file.csv @~;
```

4

Table Stages

Automatically created for each table. Accessed with `@%table_name`. Optimized for operations specific to a single table.

PUT file:///path/to/local/data.csv @%my_table;

External Stages with S3: Deep Dive

Key Configuration Parameters

- **URL:** S3 bucket path (required)
- **CREDENTIALS:** AWS access methods
- **ENCRYPTION:** Client-side encryption options
- **STORAGE_INTEGRATION:** Recommended for enterprise security

Best Practice: Use storage integrations instead of embedding credentials to maintain security and simplify IAM management.

Example S3 Stage Configuration

```
CREATE STORAGE INTEGRATION s3_int TYPE =
EXTERNAL_STAGE STORAGE_PROVIDER = S3 ENABLED = TRUE
STORAGE_AWS_ROLE_ARN =
'arn:aws:iam::001234567890:role/snowflake_role'
STORAGE_ALLOWED_LOCATIONS =
('s3://mybucket/path/');CREATE STAGE ext_stage
STORAGE_INTEGRATION = s3_int URL =
's3://mybucket/path/' FILE_FORMAT = (TYPE = 'CSV'
FIELD_DELIMITER = '|');
```



File Format Options and Best Practices

CSV/TSV

Widely compatible, but larger file sizes. Best for simple data structures.

```
TYPE = 'CSV', FIELD_DELIMITER = ',', SKIP_HEADER = 1
```

JSON

Excellent for semi-structured data. Supports nested structures. Use with VARIANT columns.

```
TYPE = 'JSON', STRIP_OUTER_ARRAY = TRUE
```

Parquet

Recommended for production. Columnar format with built-in compression. Up to 4x more efficient than CSV.

```
TYPE = 'PARQUET'
```

Avro/ORC

Row-based (Avro) and columnar (ORC) formats with schema evolution support. Good for Hadoop ecosystem integration.

PUT and GET Commands for File Transfer

PUT Command

Uploads local files to a Snowflake stage.

```
PUT file:///path/to/local/file.csv @my_stage
AUTO_COMPRESS = TRUE  OVERWRITE = TRUE;
```

Key features:

- Automatically compresses files during upload
- Parallel uploads for large files
- Supports pattern matching
- Can be executed from SnowSQL CLI or connectors

GET Command

Downloads files from a Snowflake stage to local storage.

```
GET @my_stage/data/file.csv
file:///path/to/local/directory/  DECOMPRESS = TRUE;
```

Key features:

- Automatically decompresses files during download
- Parallel downloads for large files
- Supports pattern matching
- Can be executed from SnowSQL CLI or connectors

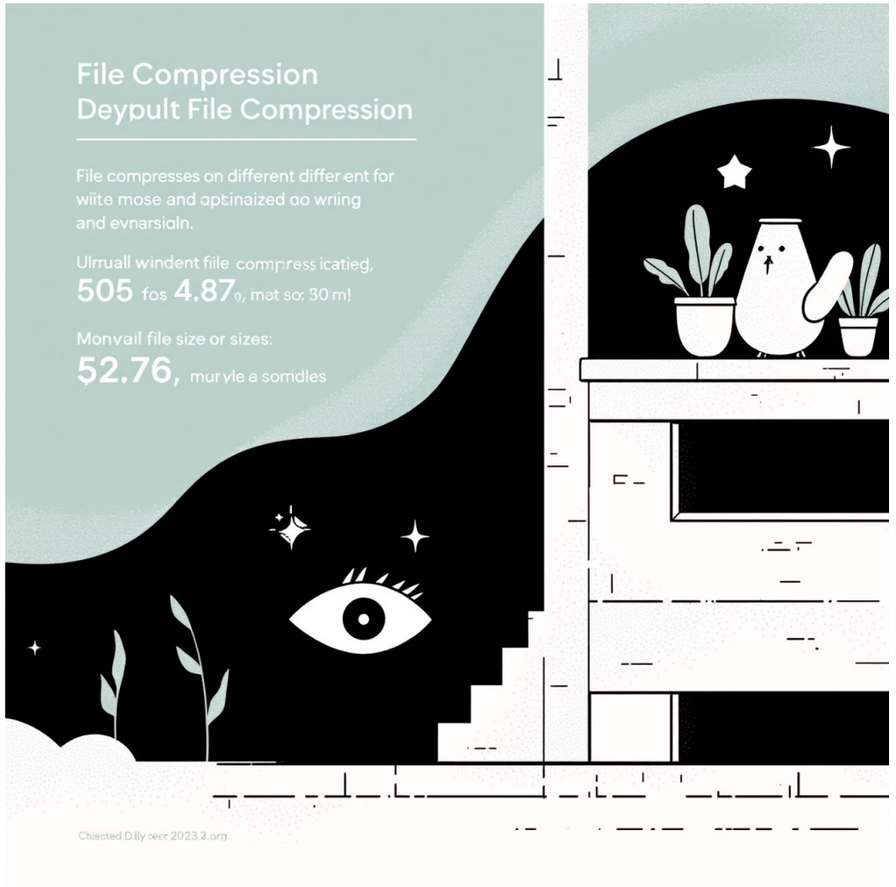
PUT and GET are client-side commands that establish a secure connection between your local environment and Snowflake. They're ideal for working with smaller files or when you need direct file transfer without using cloud storage services.

Compression Options for Staged Files

Snowflake supports multiple compression formats for staged files, which can significantly reduce storage costs and improve load performance.

Supported Compression Types

- **GZIP:** Widely supported, good compression ratio (default)
- **BZ2:** Better compression than GZIP, but slower
- **BROTLI:** Excellent balance of speed and compression
- **ZSTD:** **Recommended for production**, best overall performance
- **DEFLATE:** Similar to GZIP, different algorithm
- **RAW_DEFLATE:** DEFLATE without headers
- **NONE:** No compression, only for testing



In our benchmarks, ZSTD achieved 20% better compression than GZIP while being 30% faster for large datasets.



Advanced Stage Operations

List Files in Stage

```
LIST @my_stage PATTERN =
'.*parquet';
```

Remove Files from Stage

```
REMOVE @my_stage/old_data/;
```

Copy Data from Stage to Table

```
COPY INTO my_table FROM
@my_stage/data/ PATTERN =
'.*parquet'FILE_FORMAT = (TYPE
= PARQUET);
```

Unload Data to Stage

```
COPY INTO @my_stage/export/
FROM my_tableFILE_FORMAT =
(TYPE = PARQUET)HEADER = TRUE;
```

Stage Monitoring

Query metadata with:

```
SELECT * FROM
information_schema.filesWHERE
stage_name = 'MY_STAGE'AND
stage_schema = 'MY_SCHEMA';
```

Enterprise Implementation Patterns

Multi-Environment Architecture

Create separate stages for development, testing, and production environments to ensure isolation. Use consistent naming conventions:

```
dev_raw_stage, test_raw_stage, prod_raw_stage
```

Landing Zone Pattern

Implement a three-tiered staging approach:

1. **Raw Stage:** Initial landing for unmodified files
2. **Validated Stage:** Files that passed validation
3. **Archive Stage:** Historical file retention

Integration with AWS Services

Connect stages with S3 event notifications, Lambda functions, and Step Functions to create automated workflows:

- S3 → Lambda → Snowpipe → Snowflake Tables
- Snowflake Tasks → External Stages → S3 → Other AWS Services

Performance Optimization for Stages

Key Performance Factors

- **File Size:** Aim for 100-250MB compressed files. Too small (<10MB) or too large (>1GB) files reduce performance.
- **Partitioning:** Organize stage files by date/region to enable pattern-based loading and pruning.
- **Compression:** Always compress files. ZSTD offers the best balance of speed and size.
- **File Format:** Parquet outperforms CSV by 3-5x for complex datasets.
- **Warehouse Sizing:** Match your warehouse size to your data volume. More data requires larger warehouses.
- **Parallel Loading:** Split large datasets into multiple files to enable parallel loading.



By implementing these optimizations, one client reduced their daily load times from 4 hours to 20 minutes while reducing storage costs by 40%.

Key Takeaways

Choose the Right Stage Type

External stages with S3 for enterprise workloads and persistence.

Internal stages for ephemeral or smaller datasets.

Optimize File Formats

Use Parquet with ZSTD compression for production workloads.

Consider your data structure when selecting formats.

Secure Your Data Pipeline

Implement storage integrations, avoid hardcoded credentials, and enforce least privilege access to stages.

Follow Size Guidelines

Maintain 100-250MB file sizes and implement partitioning strategies for optimal loading performance.

Next Steps

- Audit your current stage configurations against these best practices
- Implement a standardized stage naming convention across environments
- Consider automating your data pipelines with Snowpipe for continuous loading